Laboratorio Docker MQTT-Postgres

Salomé Zuluaga Sánchez, Sara Escobar Sánchez

Facultad de Ingeniería, Universidad Autónoma de Manizales
Manizales, Colombia

I. INTRODUCCIÓN Y OBJETIVOS

El uso de contenedores ha transformado la forma en que se desarrollan, implementan y mantienen las aplicaciones, ya que permite ejecutar servicios de manera aislada y facilitar su configuración en diferentes entornos. En este contexto, Docker es una plataforma de código abierto que permite crear, ejecutar y administrar contenedores, ofreciendo una alternativa ligera y flexible para desplegar aplicaciones sin necesidad de utilizar máquinas virtuales completas. Su funcionamiento se basa en características del kernel de Linux, como los grupos de control (*cgroups*) y los espacios de nombres (*namespaces*), que permiten administrar los recursos del sistema y mantener separados los procesos que se ejecutan en cada contenedor [1].

En esta práctica se aplican estos conceptos mediante la implementación de una canalización de datos para un sistema de Internet de las Cosas (IoT). Para ello, se configuran tres servicios conectados a través de una red virtual de tipo *bridge*: un broker MQTT Eclipse Mosquitto, una base de datos PostgreSQL 18 y un cliente desarrollado en Python. Estos servicios permiten recibir datos de temperatura mediante el protocolo MQTT, basado en un modelo de publicación y suscripción, y almacenarlos en la base de datos utilizando las bibliotecas Paho-MQTT y psycopg2.

OBJETIVO GENERAL

Construir y validar de extremo a extremo una canalización de datos IoT sobre Docker, que reciba lecturas de temperatura por MQTT y las almacene de forma persistente en PostgreSQL.

OBJETIVOS ESPECÍFICOS

- Implementar un entorno de servicios contenerizados con Docker, configurando un broker MQTT con autenticación, una base de datos PostgreSQL 18 y una red bridge para permitir la comunicación entre los servicios.

- Desarrollar y contenerizar un cliente en Python que reciba mensajes MQTT en formato JSON, procese los datos de temperatura y los almacene en tablas de PostgreSQL, garantizando la persistencia de la información.

- Verificar el funcionamiento de la dirección de datos mediante la publicación de mensajes de prueba, la inspección de los registros almacenados y la identificación de posibles errores de comunicación, autenticación y persistencia.

II. METODOLOGÍA

Para el desarrollo e implementación de esta arquitectura IoT orientada a la recolección de datos de sensores, estructuramos el trabajo en contenedores Docker, lo que nos permitió aislar los servicios y asegurar que se comunicaran de forma privada.

1. Preparación del entorno y estructura del proyecto

Lo primero fue definir una red tipo "bridge" definida por el usuario llamada appnet,una red bridge de Docker es una red virtual privada que se crea dentro del propio equipo y a la que se conectan los contenedores,estos unidos a una misma red bridge pueden comunicarse libremente entre sí.
Esto fue clave para que nuestros contenedores pudieran encontrarse por nombre en lugar de depender de direcciones IP estáticas. Organizamos los directorios del proyecto separando las carpetas para el broker, la base de datos y el código del cliente Python. También centralizamos las credenciales en un archivo .env para no dejar contraseñas expuestas de forma insegura.
Se verificó la existencia del puente con el listado de redes.

```
saraesc@saraserver:~$ docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
6c5a97bada84   appnet    bridge    local
b36b494482cf   bridge    bridge    local
3c482da710cc   host      host      local
7895bb76c6fc   none      null      local
```

Img 1. Listado de redes en el docker

2. Despliegue del broker Mosquitto

Configuramos el broker MQTT para recibir los datos de temperatura. Para hacerlo más seguro, desactivamos las conexiones anónimas desde el archivo mosquitto.conf y generamos un archivo de contraseñas encriptadas usando la herramienta interna mosquitto_passwd. Luego, levantamos el contenedor mapeando el puerto 1883 hacia el host para permitir que dispositivos externos puedan publicar, y

montamos volúmenes locales para no perder la configuración ni los logs si el contenedor se apaga.

Para comprobar que nuestra configuración quedó correctamente aplicada, realizamos una prueba de inyección. Primero, verificamos en los registros del contenedor que el servicio iniciara sin errores, observando el mensaje mosquitto version 2.1.2 running. Inmediatamente después, forzamos la publicación de un mensaje de prueba ("hi") en un tópico cualquiera (test) omitiendo intencionalmente el envío de credenciales. Como esperábamos, el broker interceptó la petición y la rechazó, arrojando el error Connection error: Connection Refused: not authorised. Esto nos dio la certeza de que nuestro nodo central está protegido contra publicaciones maliciosas o no identificadas.

```
1792615963: mosquitto version 2.1.2 starting
1792615963: Config loaded from /mosquitto/config/mosquitto.conf.
1792615963: Bridge support available.
1792615963: Persistence support available.
1792615963: TLS support available.
1792615963: TLS-PSK support available.
1792615963: Websockets support available.
1792615963: Plugin builtin-security has registered to receive 'basic-auth' events.
1792615963: Opening ipv4 listen socket on port 1883.
1792615963: mosquitto version 2.1.2 running
1792615963: New connection from 172.18.0.3:43550 on port 1883.
1792615963: Client 172.18.0.3 [172.18.0.3:43550] disconnected: connection closed by client.
1792615965: New connection from 172.18.0.3:43560 on port 1883.
1792615965: Client 172.18.0.3 [172.18.0.3:43560] disconnected: connection closed by client.
1792615965: New connection from 172.18.0.3:33411 on port 1883.
1792615965: New client connected from 172.18.0.3:33411 as pg-client (p5, c1, k60, u'admin').
Connection error: Connection Refused: not authorised
Error: The connection was refused
saraesc@saraserver:~$
```

Img 2. Prueba de inyección en mosquitto

3. Despliegue de PostgreSQL 18

Para el almacenamiento de los datos, configuramos un contenedor con PostgreSQL en su versión 18. Básicamente, PostgreSQL es un motor de base de datos relacional y de código abierto, que en nuestra arquitectura cumple el rol de ser el destino final de la información. Esto es importante en los resultados para estructurar y guardar de forma permanente cada lectura de temperatura que procese el sistema, asociando cada dato con un identificador único (ID) y la estampa de tiempo exacta de su recepción (received_at), lo cual es vital para tener un registro ordenado y poder hacer consultas futuras sobre el historial de telemetría.

Al levantar este contenedor, y a diferencia del broker, decidimos no exponer el puerto de la base de datos (5432) al host por seguridad; solo los contenedores que estén dentro de nuestra red appnet pueden interactuar con ella. Además, mapeamos un volumen local en /var/lib/postgresql para garantizar la persistencia física de los registros, evitando que el historial de los sensores se pierda si el contenedor llega a reiniciarse o apagarse.

```
saraesc@saraserver:~$ cd iot-lab
saraesc@saraserver:~/iot-lab$ ls
 LICENSE  README.md  broker  client  database
saraesc@saraserver:~/iot-lab$ cd database
saraesc@saraserver:~/iot-lab/database$ ls
 pgdata
saraesc@saraserver:~/iot-lab/database$ cd pgdata
saraesc@saraserver:~/iot-lab/database/pgdata$ ls
 18
```

Img 3.Base de datos PostgreSQL versión 18

4. Desarrollo y contenerización del cliente

Para lograr que los datos fluyeran de forma autónoma desde el broker hasta la base de datos, implementamos un cliente a medida. Este desarrollo se dividió en la lógica principal, la gestión del arranque y su empaquetamiento:

a. Desarrollo del cliente principal (client.py):

Escribimos un script en Python utilizando la librería paho-mqtt en su versión 5 para suscribirnos al tópico sensors/temp. La lógica de este código está diseñada para interceptar los mensajes y decodificar su carga útil asumiendo que vienen en formato JSON (por ejemplo, {"temp": 25.5}). Una de las características más prácticas que le implementamos fue la creación dinámica de tablas: cada vez que llega un mensaje, el código adapta el nombre del tópico (cambiando la barra / por un guion bajo _ para obtener sensors_temp) y verifica si la tabla existe en PostgreSQL. Si es la primera vez que se publica en ese tópico, el cliente crea automáticamente la tabla con sus respectivas columnas (un ID autoincremental, el valor de temperatura y la marca de tiempo received_at). Finalmente, el script inserta el valor abriendo una conexión rápida a la base de datos.

```
saraesc@saraserver:~/iot-lab/client$ cd src
saraesc@saraserver:~/iot-lab/client/src$ ls
client.py  wait_for.py
saraesc@saraserver:~/iot-lab/client/src$ cat client.py
import os
import json
import psycopg2
import paho.mqtt.client as mqtt

# Configuración desde el archivo .env
MQTT_HOST = os.environ["MQTT_HOST"]
MQTT_PORT = int(os.environ["MQTT_PORT"])
MQTT_USER = os.environ["MQTT_USER"]
MQTT_PASSWORD = os.environ["MQTT_PASSWORD"]
MQTT_TOPIC = os.environ["MQTT_TOPIC"]

PG_HOST = os.environ.get("POSTGRES_HOST", "postgres")
PG_PORT = os.environ.get("POSTGRES_PORT", "5432")
PG_DB = os.environ["POSTGRES_DB"]
PG_USER = os.environ["POSTGRES_USER"]
PG_PASSWORD = os.environ["POSTGRES_PASSWORD"]

def table_name_for_topic(topic: str) -> str:
    return topic.strip("/").replace("/", "_")

def ensure_table(cur, table: str):
    cur.execute(f"""
        CREATE TABLE IF NOT EXISTS {table} (
            id SERIAL PRIMARY KEY,
            temp DOUBLE PRECISION,
            received_at TIMESTAMPTZ DEFAULT NOW()
        );
    """)

def insert_reading(cur, table: str, payload: dict):
    cur.execute(
        f'INSERT INTO "{table}" (temp) VALUES (%s)',
        (payload.get("temp"),)
    )

def on_connect(client, userdata, flags, rc, properties=None):
    print(f"[mqtt] connected rc={rc}, subscribing to {MQTT_TOPIC}")
    client.subscribe(MQTT_TOPIC)

def on_message(client, userdata, msg):
    topic = msg.topic
    table = table_name_for_topic(topic)
    try:
        payload = json.loads(msg.payload.decode())
```

Img 4.Codigo del cliente

b. Gestión de dependencias en el arranque (wait_for.py):

Al trabajar en un entorno con múltiples contenedores que inician al mismo tiempo, nos enfrentamos al problema de que el cliente de Python podía intentar conectarse a la base de datos o al broker antes de que estos estuvieran totalmente listos. Para evitar errores de conexión al arrancar, desarrollamos un script auxiliar llamado wait_for.py. Utilizando únicamente la librería estándar de Python (socket), este código se encarga de "hacer ping" a los puertos TCP de Mosquitto (1883) y PostgreSQL (5432). El script se queda en un ciclo de espera y solo cede la ejecución al cliente principal (client.py) cuando comprueba que ambos servicios están respondiendo exitosamente.

```
saraesc@saraserver:~/iot-lab/client/src$ cat wait_for.py
import os
import socket
import time
import sys

DEPS = [
    ("postgres", os.environ.get("POSTGRES_HOST", "postgres"), int(os.environ.get("POSTGRES_PORT", "5432"))),
    ("mosquitto", os.environ.get("MQTT_HOST", "mosquitto"), int(os.environ.get("MQTT_PORT", "1883"))),
]

TIMEOUT = int(os.environ.get("WAIT_TIMEOUT", "120"))
INTERVAL = int(os.environ.get("WAIT_INTERVAL", "2"))

def reachable(host: str, port: int) -> bool:
    try:
        with socket.create_connection((host, port), timeout=2):
            return True
    except OSError:
        return False

def main():
    deadline = time.time() + TIMEOUT
    while time.time() < deadline:
        still_down = [name for name, h, p in DEPS if not reachable(h, p)]
        if not still_down:
            print("[wait] all dependencies reachable")
            break
        print(f"[wait] waiting on: {', '.join(still_down)}")
        time.sleep(INTERVAL)
    else:
        if still_down:
            print(f"[wait] TIMEOUT still down: {still_down}")
        sys.exit(1)

    os.execvp("python", ["python", "-u", "client.py"])

if __name__ == "__main__":
    main()
```

Img 5. Validación existencia del wait_for.py

5. Prueba de integración

Para comprobar el funcionamiento global del sistema y verificar cómo respondía nuestro código intermediario, realizamos una inyección de datos simulando un sensor externo. Durante este proceso, mantuvimos un monitoreo en tiempo real de los registros del cliente Python ejecutando el comando docker logs -f mqtt-pg-client.

Inicialmente, el registro confirma un arranque seguro con el mensaje [wait] all dependencies reachable, lo que demuestra que nuestro script auxiliar retuvo el inicio del cliente hasta que PostgreSQL y Mosquitto estuvieron totalmente listos para recibir conexiones. Justo después, observamos el éxito principal de la prueba: el cliente intercepta la lectura simulada y confirma tanto la creación dinámica de la tabla como la persistencia del dato con la línea [db] stored in table 'sensors_temp'. Finalmente, la última sección del log revela un comportamiento muy valioso para aplicaciones reales, ya que el cliente entra en estado de espera ([wait] waiting on: postgres) y luego se reconecta automáticamente.

```
saraesc@saraserver:~$ docker logs -f mqtt-pg-client
[wait] all dependencies reachable
/app/client.py:66: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
  client = mqtt.Client(client_id="pg-client", protocol=mqtt.MQTTv5)
[mqtt] connected rc=Success, subscribing to sensors/temp
[mqtt] sensors/temp -> {'temp': 25.5}
[db] stored in table 'sensors_temp'
[wait] waiting on: postgres
[wait] all dependencies reachable
/app/client.py:66: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
  client = mqtt.Client(client_id="pg-client", protocol=mqtt.MQTTv5)
[mqtt] connected rc=Success, subscribing to sensors/temp
[wait] waiting on: postgres
[wait] all dependencies reachable
/app/client.py:66: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
  client = mqtt.Client(client_id="pg-client", protocol=mqtt.MQTTv5)
[mqtt] connected rc=Success, subscribing to sensors/temp
```

Img 6.Prueba de integración

III. RESULTADOS

Para comprobar el funcionamiento global del sistema y verificar cómo respondía nuestro código intermediario, realizamos una inyección de datos simulando un sensor externo. Durante este proceso, revisamos los registros más recientes del cliente Python ejecutando el comando docker logs mqtt-pg-client --tail 10.

Al analizar esta salida, se puede evidenciar de forma muy clara el ciclo de la arquitectura. En la primera parte del registro,el cliente entró en estado de espera ([wait] waiting on: postgres) y luego retomó su ejecución al confirmar que los servicios estaban listos[wait] all dependencies reachable). Finalmente, en las últimas dos líneas observamos el éxito definitivo de la prueba: el cliente intercepta la lectura simulada ({'temp': 25.5}) y confirma tanto la creación dinámica de la tabla como la persistencia del dato en la base de datos PostgreSQL con la línea [db] stored in table 'sensors_temp'.

```
saraesc@saraserver:~$ docker logs mqtt-pg-client --tail 10
/app/client.py:66: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
  client = mqtt.Client(client_id="pg-client", protocol=mqtt.MQTTv5)
[mqtt] connected rc=Success, subscribing to sensors/temp
[wait] waiting on: postgres
[wait] all dependencies reachable
/app/client.py:66: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
  client = mqtt.Client(client_id="pg-client", protocol=mqtt.MQTTv5)
[mqtt] connected rc=Success, subscribing to sensors/temp
[mqtt] sensors/temp -> {'temp': 25.5}
[db] stored in table 'sensors_temp'
saraesc@saraserver:~$
```

Img 7. Funcionamiento del sistema completo.

Para confirmar físicamente que la persistencia de los datos fue exitosa, ingresamos al contenedor de PostgreSQL y realizamos una consulta directa a la tabla que el cliente Python generó automáticamente mediante el siguiente comando, docker exec -it postgres psql -U app_user -d iot_db -c "SELECT * FROM sensors_temp;". A continuación se muestra la salida obtenida en la terminal de la base de datos.

```
 id | temp |          received_at
----+------+-------------------------------
  1 | 25.5 | 2026-10-14 21:50:47.203227+00
  2 | 25.5 | 2026-10-21 21:44:30.754463+00
(2 rows)

saraesc@saraserver:~$
```

Img 8. Base de datos.

Como se puede observar en la salida, la base de datos estructuró correctamente la información en las columnas diseñadas (id, temp y received_at). Se puede evidenciar que la consulta nos arrojó dos registros distintos, evidenciando cómo el sistema asigna automáticamente un identificador incremental (ID 1 y 2) para cada evento. Además, podemos ver que el sistema almacenó una lectura antigua del 14 de octubre junto con la nueva inyección del 21 de octubre, asignándoles a ambas estampas de tiempo extremadamente precisas.

IV. CONCLUSIONES

La implementación de los contenedores Docker y la creación de una red independiente permitieron mejorar la seguridad del sistema IoT, ya que los servicios se mantienen aislados y se controla el acceso a la información. Además, al configurar Mosquitto para rechazar conexiones anónimas y evitar la exposición directa de PostgreSQL, se reduce el riesgo de que personas o dispositivos no autorizados accedan al sistema o envíen datos sin permiso.

El desarrollo del cliente intermediario en Python facilitó la comunicación entre MQTT y PostgreSQL, permitiendo recibir los datos de los sensores y almacenarlos en la base de datos. Asimismo, la creación automática de tablas según los tópicos recibidos hace que el sistema sea más flexible, ya que permite agregar nuevos sensores o variables sin tener que realizar tantos cambios manuales en la configuración.

Por último, las pruebas de integración permitieron comprobar que el sistema puede manejar algunos problemas de conexión entre sus componentes sin detenerse inmediatamente. Gracias al script de espera (wait_for.py) y al manejo de errores en Python, el cliente intermediario puede esperar a que los servicios estén disponibles y gestionar las interrupciones temporales, lo que contribuye a que el sistema sea más estable y confiable.

V. REFERENCIAS

[1] https://www.redhat.com/es/topics/containers/what-is-docker

[2] Repositorio de GITHUB

https://github.com/saraescobarsg/iot-lab